

BAB I

PENDAHULUAN

1.1 Latar Belakang

Evolusi serangan *ransomware* tidak lagi sekadar mengenkripsi data operasional, melainkan secara agresif menargetkan repositori cadangan (*backup*) untuk menghilangkan opsi pemulihan korban. Cybersecurity and Infrastructure Security Agency (CISA) melaporkan bahwa dalam rentang tahun 2021 hingga 2023, eskalasi ancaman siber global mencapai lebih dari 1,4 miliar upaya serangan ransomware (CISA, 2024). Sejalan dengan tingginya volume serangan siber global, publikasi riset Sophos State of Ransomware (2023) mengungkapkan bahwa 94% insiden secara spesifik menargetkan penghancuran data cadangan, dengan lebih dari 70% serangan berhasil merusak repositori secara fatal. Tingkat keberhasilan penghancuran data terjadi karena mayoritas arsitektur penyimpanan cadangan masih bersifat mutable (dapat diubah). Akibatnya, saat penyerang menguasai server aplikasi, mereka menyalahgunakan jalur otentikasi sah untuk menghapus atau menimpa data cadangan secara permanen. Tanpa mekanisme pertahanan pemisah domain kontrol aplikasi dari integritas fisik data, organisasi kehilangan garis pertahanan terakhir.

Menghadapi ancaman manipulasi repositori cadangan, penelitian yang dilakukan bertujuan membangun arsitektur penyimpanan yang mampu mencegah penghapusan data melalui jalur aplikasi yang diretas. Solusi standar pencegahan modifikasi data bergantung pada teknologi *Write-Once-Read-Many* (WORM). Walaupun WORM mencegah penghapusan data dalam periode tertentu, penerapan WORM memicu inefisiensi kapasitas penyimpanan. Perubahan kecil pada berkas berukuran besar memaksa sistem WORM konvensional menyimpan ulang seluruh berkas sebagai objek baru, sehingga memicu lonjakan kebutuhan kapasitas (*data bloat*). Fokus utama penelitian yang dilakukan adalah mengimplementasikan mekanisme efisiensi penyimpanan tanpa mengorbankan integritas fisik data.

Pendekatan untuk menjembatani kebutuhan keamanan dan efisiensi mengarah pada pengembangan sistem *Immutable Object Storage*. Sistem mengintegrasikan *Content-Addressable Storage* (CAS) dengan algoritma *Fast*

Content-Defined Chunking (FastCDC). Algoritma FastCDC memecah berkas menjadi potongan-potongan (*chunk*) berdasarkan isi konten, sehingga basis data hanya menyimpan *chunk* berstatus baru melalui mekanisme deduplikasi. Pendekatan deduplikasi memungkinkan penerapan properti *immutability* ketat untuk memitigasi *ransomware*, sekaligus menjaga efisiensi kapasitas penyimpanan agar tetap optimal.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah pada penelitian yang dilakukan adalah bagaimana mengimplementasikan sistem penyimpanan *Immutable Object Storage* berbasis CAS dan algoritma FastCDC untuk mitigasi serangan *ransomware*.

1.3 Ruang Lingkup

Penelitian pengembangan *Immutable Object Storage* mencakup berbagai hal, sebagai berikut:

- a. Penelitian hanya menggunakan teknologi *Content-Addressable Storage* (CAS) dengan algoritma *Fast Content-Defined Chunking* (FastCDC) untuk mencapai efisiensi penyimpanan melalui deduplikasi dan immutabilitas logis.
- b. Sistem yang dikembangkan mencakup fitur manajemen retensi data dengan mekanisme *soft delete* tanpa mencakup proses penghapusan fisik (*Garbage Collection*). Untuk keamanan dasar operasional, sistem hanya menerapkan *rate limiting* dan batasan ukuran maksimal objek, tanpa mencakup manajemen kuota pengguna *multi-tenant*.
- c. Penelitian hanya difokuskan pada pengujian di lingkungan *single-node* dengan isolasi keamanan berbasis pemisahan hak akses sistem operasi melalui *Unix Domain Socket* (UDS), sehingga tidak mencakup isolasi fisik jaringan antar-server, fitur *High Availability*, maupun replikasi terdistribusi.
- d. Modul pengunduhan data (*retrieval*) pada sistem di tahap ini mengizinkan antarmuka API memiliki akses baca langsung (*Read-Only Mount*) ke

direktori penyimpanan fisik, dan belum menggunakan lapisan isolasi proksi baca (*Read-Proxy*).

- e. Prototipe sistem dikembangkan untuk memvalidasi fungsi perlindungan data dari manipulasi peretas, mengukur rasio deduplikasi, serta menguji ketahanan pemulihan sistem (*crash consistency*), dengan arsitektur yang secara tegas memisahkan entitas logis aplikasi dan penyimpanan.

1.4 Tujuan & Manfaat

Tujuan penelitian adalah mengembangkan prototipe sistem *Immutable Object Storage* yang efisien dan aman serta mampu menjamin integritas data cadangan saat peladen aplikasi mengalami peretasan. Penelitian juga bertujuan untuk mengimplementasikan algoritma *Fast Content-Defined Chunking* (FastCDC) dalam arsitektur *Content-Addressable Storage* (CAS), memvalidasi ketahanan mekanisme konsistensi metadata, serta memaksimalkan efisiensi penyimpanan melalui deduplikasi tingkat *chunk*, sehingga risiko kehilangan data akibat kegagalan sistem atau manipulasi *ransomware* dapat dicegah.

Manfaat penelitian pengembangan *Immutable Object Storage* untuk mitigasi *ransomware* adalah sebagai berikut:

- a. Meningkatkan perlindungan data cadangan organisasi dengan memastikan penutupan celah manipulasi melalui jalur aplikasi yang diretas, sekaligus memisahkan otoritas modifikasi data secara permanen.
- b. Mengurangi inefisiensi infrastruktur penyimpanan data arsip *Write-Once-Read-Many* (WORM), karena implementasi mekanisme deduplikasi pengurang redundansi memungkinkan pemakaian kapasitas penyimpanan yang optimal.
- c. Mempermudah pengembang sistem keamanan siber dalam menerapkan pertahanan berlapis pada sistem penyimpanan kritis, sehingga prototipe dapat menjadi model arsitektur referensi yang andal.

1.5 Sistematika

Adapun sistematika penulisan laporan Proyek Utama Informatika disusun dalam beberapa bab, dapat dijelaskan sebagai berikut:

Bab I Pendahuluan

Bab ini menguraikan penjelasan tentang latar belakang masalah kerentanan data cadangan, rumusan masalah, batasan ruang lingkup sistem, serta tujuan dan manfaat penelitian pengembangan *Immutable Object Storage*.

Bab II Tinjauan Pustaka dan Teori

Bab ini membahas hasil penelitian sebelumnya yang relevan dengan topik mitigasi *ransomware* dan penyimpanan data. Pada landasan teori, dibahas konsep dasar yang mendukung penelitian, meliputi teori *Content-Addressable Storage* (CAS), algoritma *Fast Content-Defined Chunking* (FastCDC), prinsip kriptografi *hashing*, serta mekanisme keamanan sistem penyimpanan WORM.

Bab III Metode Penelitian

Bab ini memberikan penjelasan menyeluruh tentang metodologi penelitian yang digunakan untuk menyelesaikan masalah. Penjelasan mencakup kerangka penelitian, arsitektur sistem yang memisahkan *boundary* aplikasi dan penyimpanan, perancangan model data untuk *chunk* dan *manifest*, serta analisis kebutuhan fungsional sistem.

Bab IV Produk

Bab ini memaparkan hasil implementasi prototipe sistem *Immutable Object Storage*, pembahasan hasil pengujian terhadap skenario serangan *ransomware*, serta rencana pengembangan fitur lanjutan seperti *concurrent GC* dan audit log pada tahap Proyek Profesional dan Tugas Akhir.

Bab V Simpulan

Bagian penutup membahas kesimpulan dari hasil pengembangan *baseline* sistem serta saran untuk perbaikan di masa mendatang.

Daftar Pustaka

Daftar pustaka berisi sitasi dari berbagai referensi yang digunakan dalam penyusunan laporan, yang meliputi jurnal ilmiah, prosiding konferensi, dan dokumentasi teknis terkait.

BAB II

TINJAUAN PUSTAKA DAN TEORI

2.1 Tinjauan Pustaka

Tinjauan pustaka menguraikan berbagai penelitian terdahulu yang relevan dengan pengembangan arsitektur penyimpanan dan mitigasi *ransomware*. Penulis mengintegrasikan temuan-temuan sebelumnya sebagai landasan teoretis untuk membangun kebaruan rancangan sistem pada penelitian yang dilakukan.

Penelitian Caporaso, dkk. (2024) membahas pengembangan sistem berkas *write-once* bernama VaultFS sebagai solusi mitigasi serangan *ransomware*. Peneliti menjadikan sistem berkas tingkat *kernel* pada lingkungan Linux sebagai objek penelitian dan memodifikasinya untuk menolak operasi penulisan ulang. Peneliti mengimplementasikan kebijakan *write-once* pada lapisan VFS (*Virtual File System*). Hasil penelitian menunjukkan bahwa pemisahan domain kontrol pada tingkat sistem berkas merupakan garis pertahanan krusial saat terjadi peretasan. Akun administratif yang diretas tetap tidak dapat memodifikasi data yang terkunci. Penelitian mengangkat permasalahan terkait ketiadaan mekanisme proteksi lapisan sistem berkas yang independen dari jalur autentikasi aplikasi. Keterbatasan penelitian terdahulu terletak pada fokus proteksi tingkat sistem berkas yang belum mempertimbangkan efisiensi kapasitas penyimpanan. Penelitian menutup celah penelitian terdahulu dengan menambahkan lapisan deduplikasi berbasis *Content-Addressable Storage* (CAS). Penulis memastikan sistem menerapkan *immutability* tanpa memicu pemborosan kapasitas penyimpanan.

Berdasarkan penelitian yang dilakukan oleh Múzquiz, dkk. (2025), peneliti mengeksplorasi pemanfaatan perangkat penyimpanan *Write-Once-Read-Many* (WORM) untuk menjaga integritas data log. Peneliti menjadikan perangkat keras WORM berbasis optik yang diintegrasikan ke dalam sistem penyimpanan terbuka sebagai objek penelitian. Penelitian menerapkan metode validasi integritas berbasis perangkat keras dengan mekanisme pemeriksaan *checksum* pada tingkat fisik media. Hasil penelitian mengungkapkan bahwa validasi integritas tingkat perangkat keras efektif mencegah modifikasi tanpa izin, sehingga setiap objek tersimpan memiliki jaminan keaslian yang bersifat intrinsik. Penelitian mengangkat

permasalahan terkait kebutuhan perangkat keras khusus yang mahal dan sulit diimplementasikan pada infrastruktur komoditas. Keterbatasan penelitian terdahulu terletak pada ketergantungan terhadap media fisik WORM yang tidak dapat diterapkan secara fleksibel. Penelitian yang dilakukan menawarkan pendekatan berbeda dengan mewujudkan sifat *immutability* secara logis melalui CAS pada perangkat keras komoditas, tanpa ketergantungan pada media fisik khusus.

Udayashankar, dkk. (2025) pada penelitiannya menguji akselerasi algoritma *Fast Content-Defined Chunking* (FastCDC) menggunakan instruksi vektor perangkat keras. Peneliti mengoptimalkan implementasi FastCDC menggunakan ekstensi *Single Instruction, Multiple Data* (SIMD) pada prosesor modern sebagai objek penelitian. Penelitian menggunakan metode perbandingan throughput antara implementasi skalar dan implementasi terakselerasi vektor pada dataset berkas berukuran besar. Peneliti menyimpulkan bahwa akselerasi perangkat keras mampu meningkatkan performa pemecahan berkas secara signifikan tanpa mengorbankan rasio deduplikasi. Penelitian mengangkat permasalahan terkait latensi *chunking* yang menjadi hambatan pada sistem cadangan berskala besar. Keterbatasan penelitian terdahulu terletak pada fokus terhadap akselerasi berbasis perangkat keras eksklusif yang tidak selalu tersedia di lingkungan umum. Penelitian yang dilakukan mengadopsi algoritma FastCDC tanpa bergantung pada ekstensi SIMD khusus, sehingga implementasi dapat berjalan pada infrastruktur komoditas standar dengan tetap mempertahankan efisiensi deduplikasi untuk menangani masalah *data bloat*.

Penelitian Lv, dkk. (2025) membahas tinjauan komprehensif terhadap arsitektur basis data *key-value* berbasis *Log-Structured Merge-tree* (LSM). Peneliti mengevaluasi berbagai implementasi LSM-tree seperti RocksDB, LevelDB, dan BadgerDB pada beban kerja nyata sistem terdistribusi. Peneliti menggunakan metode survei komparatif terhadap performa tulis, baca, dan efisiensi ruang pada berbagai implementasi LSM. Peneliti menunjukkan bahwa struktur LSM memberikan performa tulis yang stabil dan sangat efisien dalam menangani jutaan referensi *chunk* pada sistem terdistribusi. Penelitian mengangkat permasalahan terkait tingginya beban operasi baca-acak pada sistem metadata berskala besar

akibat proses *compaction*. Keterbatasan penelitian terdahulu terletak pada analisis yang bersifat survei tanpa mengusulkan mekanisme baru yang secara langsung mendukung properti *immutability*. Temuan dari survei menjadi landasan bagi penelitian yang dilakukan dalam memilih BadgerDB sebagai mesin penyimpanan metadata *chunk* dan *manifest* objek pada sistem *Immutable Object Storage*.

Berdasarkan penelitian yang dilakukan oleh Kim, dkk. (2026), peneliti menganalisis evolusi perilaku *ransomware* invasif yang secara spesifik menargetkan penghancuran data *cloud repository*. Peneliti menjadikan sampel *ransomware* modern yang memanfaatkan eskalasi hak istimewa untuk menyerang layanan penyimpanan berbasis *cloud* sebagai objek penelitian. Penelitian menerapkan metode analisis perilaku statis dan dinamis terhadap varian *ransomware* pada lingkungan terisolasi. Peneliti menyarankan penggunaan mekanisme validasi berbasis format untuk mendeteksi anomali enkripsi guna memitigasi risiko *credential theft*. Penelitian mengangkat permasalahan terkait ketidakefektifan sistem deteksi berbasis pola terhadap varian *ransomware* yang terus berevolusi. Keterbatasan penelitian terdahulu terletak pada pendekatan detektif yang reaktif, bukan pada aspek pencegahan struktural. Penelitian yang dilakukan mengambil posisi komplementer dengan membangun arsitektur penyimpanan yang secara struktural mencegah kerusakan data, sehingga meskipun deteksi gagal, data cadangan tetap terlindungi dari manipulasi.

Shi, dkk. (2026) pada penelitiannya membahas komparasi performa fungsi kriptografi antara BLAKE3 dan algoritma lama pada arsitektur modern. Peneliti mengevaluasi performa fungsi *hash* SHA-256, SHA-512, MD5, dan BLAKE3 pada beban kerja komputasi intensif. Penelitian menerapkan metode pengukuran *throughput* dan latensi pada berbagai ukuran data masukan. Peneliti menyimpulkan bahwa BLAKE3 menawarkan kecepatan pemrosesan yang jauh lebih tinggi melalui paralelisasi internal berbasis pohon *Merkle*, sehingga sangat cocok untuk proses identifikasi *chunk* pada sistem deduplikasi. Penelitian mengangkat permasalahan terkait tingginya biaya komputasi fungsi *hash* konvensional yang menjadi hambatan performa sistem penyimpanan berskala besar. Keterbatasan penelitian terdahulu terletak pada konteks pengujian yang berfokus pada kecepatan komputasi

secara umum tanpa diterapkan secara konkret pada arsitektur CAS. Penelitian memanfaatkan temuan terdahulu dengan mengadopsi BLAKE3 sebagai fungsi *fingerprinting* pada setiap *chunk* untuk memastikan identitas konten yang unik dan performa *hashing* yang optimal dalam sistem *Immutable Object Storage*.

Penelitian Rahman, dkk. (2025) membahas pendekatan arsitektural untuk menjamin autentikasi dan sifat *immutability* pada data transaksi perbankan. Peneliti menjadikan sistem perbankan digital dengan kebutuhan keamanan data transaksi tingkat tinggi sebagai objek penelitian. Penelitian mengusulkan metode rancangan arsitektur multi-lapis yang memisahkan lapisan autentikasi, validasi, dan penyimpanan secara hierarkis. Penelitian menunjukkan bahwa arsitektur multi-lapis efektif mengamankan data dari serangan manipulasi internal. Penelitian mengangkat permasalahan terkait serangan dari dalam (*insider threat*) yang memiliki akses sah namun menyalahgunakan wewenang untuk memanipulasi rekaman transaksi. Keterbatasan penelitian terdahulu terletak pada domain yang terbatas pada sistem perbankan tanpa mempertimbangkan skenario serangan eksternal berbasis *ransomware*. Prinsip pemisahan wewenang pada penelitian terdahulu relevan dengan sistem yang dikembangkan penulis, khususnya dalam merancang mekanisme pemisahan domain kontrol aplikasi dari akses langsung ke penyimpanan fisik.

Berdasarkan penelitian yang dilakukan oleh Higuchi, dkk. (2026), peneliti mengkaji dampak *hook point* sistem berkas terhadap rasio pencadangan secara *real-time*. Peneliti mengevaluasi sistem berkas XFS pada *kernel* Linux yang dilengkapi mekanisme pencegahan operasi *file-open hook* untuk pencadangan otomatis. Penelitian menggunakan metode pengukuran rasio berkas tercadangan pada skenario serangan enkripsi massal. Penelitian membuktikan bahwa menunda penulisan fisik dan mencegah operasi berkas memberikan waktu tambahan bagi sistem untuk memitigasi kerusakan akibat enkripsi *ransomware*. Penelitian mengangkat permasalahan terkait tingginya risiko kehilangan data pada jendela waktu antara modifikasi berkas dan proses pencadangan. Keterbatasan penelitian terdahulu terletak pada pendekatan yang bergantung pada kecepatan respons sistem, bukan pada struktur penyimpanan yang imun terhadap penghapusan.

Penelitian melengkapi pendekatan terdahulu dengan menyediakan repositori tujuan pencadangan yang bersifat *immutable*, sehingga data yang berhasil tercadangkan tidak dapat dihancurkan oleh *ransomware*.

Pipalani, dkk. (2025) pada penelitiannya mengeksplorasi ekosistem pengujian repositori berbasis Go. Peneliti mengevaluasi kualitas dan kelengkapan pengujian unit (*unit test*) pada kode sumber Go yang dikembangkan oleh komunitas daring. Penelitian menerapkan metode *fine-tuning* model bahasa besar (LLM) terhadap dataset pengujian Go untuk menghasilkan *unit test* secara otomatis. Peneliti menekankan bahwa struktur konkurensi bawaan pada Go jauh lebih efisien dalam menangani operasi *input/output* asinkron dibandingkan bahasa pemrograman tradisional. Penelitian mengangkat permasalahan terkait rendahnya cakupan pengujian otomatis pada proyek Go berskala besar. Keterbatasan penelitian terdahulu terletak pada konteks yang berfokus pada pengujian otomatis, bukan pada performa atau keandalan sistem penyimpanan. Temuan karakteristik Go sebagai bahasa yang unggul dalam konkurensi mendukung keputusan penelitian yang dilakukan untuk mengimplementasikan sistem *Immutable Object Storage* menggunakan Go guna menangani operasi baca-tulis bersamaan secara aman dan efisien.

Penelitian Xu, dkk. (2023) membahas pemodelan prediksi berbasis *machine learning* untuk memperkirakan performa baca dan tulis pada berbagai subsistem penyimpanan. Peneliti mengevaluasi subsistem penyimpanan heterogen yang mencakup HDD, SSD, dan NVMe di bawah beban kerja campuran. Penelitian menggunakan metode pemodelan regresi terhadap metrik latensi dan *throughput* berdasarkan karakteristik beban kerja. Peneliti membuktikan bahwa optimalisasi penempatan data berdasarkan pola akses menjamin sistem tetap stabil meskipun menerima beban kerja yang tinggi. Penelitian mengangkat permasalahan terkait sulitnya memprediksi degradasi performa saat subsistem penyimpanan menerima beban kerja campuran secara bersamaan. Keterbatasan penelitian terdahulu terletak pada fokus prediksi performa yang tidak membahas mekanisme perlindungan integritas data. Metodologi pengujian beban yang digunakan oleh Xu dkk. (2023) menjadi acuan bagi penelitian yang dilakukan dalam merancang skenario pengujian

throughput dan stabilitas sistem *Immutable Object Storage* di bawah beban kerja tinggi. Tabel 2.1 menyajikan perbandingan sumber pustaka primer.

Tabel 2.1 Sumber Pustaka Primer

No	Judul	Penulis (tahun)	Metode/Alat	Hasil/Kesimpulan
1	<i>VaultFS: Write-once Software Support at the File System Level Against Ransomware Attacks</i>	Pasquale Caporaso, Giuseppe Bianchi, Francesco Quaglia (2024)	VaultFS	Pemisahan kontrol penyimpanan efektif mencegah manipulasi cadangan.
2	<i>The Reverse File System: Towards open cost-effective secure WORM storage</i>	Gorka Guardiola Múzquiz, Juan González-Gómez, Enrique Soriano-Salvador (2025)	WORM	Perangkat keras write-once mengamankan data dari penghapusan massal.
3	<i>Accelerating Data Chunking in Deduplication Systems using Vector Instructions</i>	Sreeharsha Udayashankar, Abdelrahman Baba, Samer Al-Kiswany (2025)	FastCDC	Akselerasi vektor meningkatkan kecepatan chunking secara signifikan.
4	<i>Rethinking LSM-tree based Key-Value Stores: A Survey</i>	Yina Lv, Qiao Li, Quanqing Xu, Congming Gao, Chuanhui Yang, Xiaoli Wang, Chun Jason Xue (2025)	LSM-Tree	Struktur LSM sangat efisien untuk mengelola referensi metadata skala besar.
5	<i>Rhea: Detecting Privilege-Escalated Evasive Ransomware Attacks</i>	Beom Heyn Kim, Seok Min Hong (2026)	Analisis Ancaman	Validasi format memitigasi risiko serangan berbasis eskalasi hak istimewa.
6	<i>Trusting What You Cannot See: Auditable Fine-Tuning and Inference</i>	Shanghao Shi, Heng Jin, Chaoyu Zhang, Hexuan Yu, Ning Zhang, Y. Thomas Hou (2026)	BLAKE3	BLAKE3 memberikan performa hashing lebih cepat melalui fitur paralelisasi.
7	<i>A Multi-Layer Architecture for Trusted, Verifiable, and Immutable Data</i>	Aufa Nasywa Rahman, Bimo Sunarfri Hantono, Guntur Dharma Putra (2025)	Immutability	Arsitektur multi-lapis menjamin integritas data secara intrinsik.

Tabel 2.1 Sumber Pustaka Primer

No	Judul	Penulis (tahun)	Metode/Alat	Hasil/Kesimpulan
8	<i>Impact of File-Open Hook Points on Backup Ratio in ROFBS on XFS</i>	Kosuke Higuchi, Ryotaro Kobayashi (2026)	Real-time Backup	Intersepsi operasi berkas memberikan perlindungan tambahan masa pemulihan.
9	<i>Go-UT-Bench: A Fine-Tuning Dataset for LLM-Based Unit Test Generation in Go</i>	Yashshi Pipalani, Rajat Ghosh, Hritik Raj, Vaishnavi Bhargava, Debojyoti Dutta (2025)	Golang	Go memberikan efisiensi tinggi dalam menangani operasi asinkron.
10	<i>ML-based Modeling to Predict I/O Performance on Storage Sub-systems</i>	Yiheng Xu, Pranav Sivaraman, Hariharan Devarajan, Kathryn Mohror, Abhinav Bhatele (2023)	Analisis I/O	Prediksi beban kerja menjamin stabilitas sistem penyimpanan.

Tabel 2.1 menyajikan ringkasan perbandingan penelitian terdahulu. Penulis menyimpulkan bahwa fokus utama penelitian terdahulu belum mencakup integrasi antara mekanisme perlindungan sistem berkas yang tangguh dengan efisiensi kapasitas penyimpanan. Penelitian terdahulu cenderung mengorbankan efisiensi ruang demi mencapai tingkat keamanan tinggi atau sebaliknya. Penelitian menutup celah penelitian terdahulu dengan merancang arsitektur yang menggabungkan lapisan deduplikasi berbasis *Content-Addressable Storage* (CAS) dan *Virtual File System* (VFS). Penulis memastikan sistem mampu menerapkan *immutability* secara ketat tanpa memicu pemborosan kapasitas penyimpanan, sehingga memberikan perlindungan optimal terhadap serangan *ransomware*.

2.2 Teori

2.2.1 Sistem

Sistem adalah sekumpulan komponen yang saling berinteraksi dan bekerja sama untuk mencapai tujuan tertentu. Setiap sistem terdiri atas masukan (*input*),

proses, dan keluaran (*output*) yang terorganisasi dalam batas-batas tertentu. Sistem yang dirancang dengan baik memiliki mekanisme kontrol untuk memastikan setiap komponen berfungsi sesuai dengan spesifikasi yang ditetapkan (Whitten, dkk. 2004). Dalam penelitian yang dilakukan, sistem yang dibangun merupakan sekumpulan komponen perangkat lunak yang mencakup lapisan API, mesin deduplikasi, dan penyimpanan fisik yang berinteraksi melalui protokol terdefinisi untuk mewujudkan properti *immutability* pada data cadangan.

2.2.2 Penyimpanan Data

Penyimpanan data adalah proses merekam informasi digital pada media fisik atau logis agar dapat diambil kembali pada saat dibutuhkan. Arsitektur penyimpanan modern diklasifikasikan berdasarkan model akses, yaitu penyimpanan blok (*block storage*), penyimpanan berkas (*file storage*), dan penyimpanan objek (*object storage*). Penyimpanan objek menyimpan data sebagai unit diskrit yang dilengkapi metadata, sehingga lebih fleksibel dan skalabel dibandingkan model lainnya (Vacca, 2022). Penelitian mengadopsi model penyimpanan objek sebagai fondasi arsitektur. Sistem mengidentifikasi setiap objek secara unik berdasarkan nilai *hash* kontennya.

2.2.3 Immutability

Sifat *immutability* pada sistem penyimpanan berarti data yang telah ditulis tidak dapat diubah, ditimpa, atau dihapus selama periode retensi yang ditetapkan. Prinsip ini berbeda dari enkripsi maupun kontrol akses: alih-alih membatasi siapa yang dapat membaca atau memodifikasi data, *immutability* menolak setiap operasi penulisan ulang secara struktural, termasuk dari akun dengan hak akses administrator sekalipun. Implementasi *immutability* dapat dilakukan secara fisik melalui teknologi WORM maupun secara logis melalui arsitektur perangkat lunak seperti CAS (Silberschatz dkk., 2024). Dalam penelitian yang dilakukan, sifat *immutability* ditegakkan secara logis melalui pemisahan domain antara lapisan API dan penyimpanan fisik, sehingga tidak ada jalur yang memungkinkan penghapusan atau penimpaan *chunk* yang sudah tersimpan.

2.2.4 Write-Once-Read-Many (WORM)

WORM adalah teknologi penyimpanan yang memungkinkan data ditulis hanya satu kali ke perangkat penyimpanan, namun dapat dibaca berulang kali tanpa batas. Teknologi WORM digunakan untuk arsip kepatuhan regulasi dan perlindungan data kritis karena memberikan jaminan fisik bahwa data tidak akan mengalami modifikasi setelah disimpan. Meskipun efektif, implementasi WORM konvensional menyimpan ulang seluruh berkas setiap kali terjadi perubahan sekecil apapun, sehingga memicu pemborosan kapasitas yang dikenal sebagai *data bloat* (Silberschatz dkk., 2024). Penelitian yang dilakukan mengambil inspirasi dari prinsip WORM namun menggantikan ketergantungan pada media fisik dengan mekanisme *immutability* logis berbasis CAS, sekaligus mengatasi masalah *data bloat* melalui deduplikasi tingkat *chunk*.

2.2.5 Immutable Object Storage

Immutable Object Storage adalah sistem penyimpanan objek yang menerapkan sifat permanen pada data yang telah ditulis. Sistem mengidentifikasi setiap objek berdasarkan isi kontennya. Setelah objek tersimpan, data tidak dapat diubah, ditimpa, atau dihapus selama periode retensi yang telah ditentukan. Sistem mengunci integritas fisik data guna mencegah manipulasi oleh peretas yang telah menguasai jalur autentikasi aplikasi. Berbeda dari penyimpanan objek konvensional yang masih mengizinkan penimpaan dengan versi baru, *Immutable Object Storage* menolak operasi penghapusan atau modifikasi secara penuh pada tingkat arsitektur (Vacca, 2022). Penelitian yang dilakukan mengimplementasikan *Immutable Object Storage* dengan memisahkan proses penulisan melalui Unix Domain Socket (UDS) yang hanya menerima operasi tulis-tambah, sehingga lapisan aplikasi tidak memiliki jalur langsung untuk menghapus data yang telah tersimpan secara fisik.

2.2.6 Content-Addressable Storage (CAS)

Content-Addressable Storage (CAS) adalah model penyimpanan yang mengidentifikasi dan mengambil setiap unit data berdasarkan nilai *hash* dari isinya, bukan berdasarkan lokasi fisik atau nama berkas. Properti ini menjamin bahwa dua

objek dengan konten identik akan selalu menghasilkan alamat yang sama, sehingga sistem secara otomatis mendeteksi dan menolak penulisan duplikat. Jika isi objek berubah, nilai *hash*-nya juga berubah, sehingga CAS secara alami mendukung sifat *immutability* dan deteksi redundansi data (Buyya dkk., 2022). Penelitian mengimplementasikan CAS sebagai lapisan inti sistem. Lapisan inti mengalamatkan setiap *chunk* yang dihasilkan FastCDC berdasarkan nilai BLAKE3-nya sebelum disimpan ke direktori penyimpanan fisik.

2.2.7 Content-Defined Chunking

Deduplikasi data adalah proses menghilangkan salinan redundan dari data yang sama untuk mengoptimalkan kapasitas penyimpanan. Teknik deduplikasi dapat dilakukan pada tingkat berkas, blok dengan ukuran tetap, atau potongan berukuran variabel berdasarkan konten. *Content-Defined Chunking* (CDC) adalah teknik pemecahan berkas menjadi potongan-potongan (*chunk*) berukuran variabel berdasarkan pola konten, dengan menggeser jendela *hash* untuk mendeteksi batas pemisahan alami. Pendekatan CDC menghasilkan rasio deduplikasi yang lebih tinggi dibandingkan pemecahan berbasis ukuran tetap, karena pergeseran konten tidak mengubah identitas *chunk* yang tidak berubah (Hwang dkk., 2023). Dalam penelitian yang dilakukan, sistem menggunakan CDC sebagai mesin pemecah berkas sebelum setiap *chunk* diproses lebih lanjut oleh mesin CAS untuk deduplikasi dan penyimpanan permanen.

2.2.8 Fast Content-Defined Chunking (FastCDC)

Fast Content-Defined Chunking (FastCDC) adalah algoritma CDC generasi terbaru yang dirancang untuk memaksimalkan kecepatan pemecahan berkas tanpa mengorbankan rasio deduplikasi. FastCDC mengoptimalkan performa dengan menggunakan jendela geser berbobot dan fungsi *hash* sederhana untuk menentukan batas *chunk*, serta menerapkan kebijakan ukuran minimum dan maksimum *chunk* untuk mengendalikan distribusi ukuran. Pendekatan memungkinkan deduplikasi tingkat *chunk* yang sangat efisien dalam menghemat kapasitas penyimpanan, khususnya pada berkas yang mengalami perubahan parsial (Hwang dkk., 2023). Penelitian yang dilakukan menggunakan FastCDC sebagai tahap pertama alur

penyimpanan objek. Setiap berkas yang masuk dipecah menjadi *chunk* oleh FastCDC sebelum masing-masing *chunk* diproses oleh mesin CAS untuk pengecekan duplikasi dan penyimpanan permanen.

2.2.9 Kriptografi dan Fungsi Hash

Kriptografi adalah ilmu yang mempelajari teknik pengamanan informasi melalui transformasi matematis. Salah satu primitif kriptografi yang fundamental adalah fungsi *hash*, yaitu fungsi satu arah yang memetakan data masukan berukuran sembarang menjadi keluaran berukuran tetap yang disebut *digest*. Fungsi *hash* kriptografi memiliki tiga sifat utama yaitu resistansi terhadap praimej, resistansi terhadap tabrakan kedua, dan resistansi terhadap tabrakan bebas. Sifat-sifat ini menjamin bahwa perubahan sekecil apapun pada data masukan menghasilkan *digest* yang berbeda secara signifikan (Stallings, 2023). Dalam penelitian yang dilakukan, penulis menggunakan fungsi *hash* sebagai mekanisme penentuan identitas setiap *chunk* pada sistem CAS, sekaligus sebagai instrumen verifikasi integritas data.

2.2.10 BLAKE3

BLAKE3 adalah fungsi *hash* kriptografi generasi terbaru yang dirancang untuk kecepatan tinggi dan keamanan maksimum. BLAKE3 menggunakan struktur pohon *Merkle* yang memungkinkan pemrosesan data secara paralel pada berbagai ukuran blok, menjadikannya jauh lebih cepat daripada SHA-256 maupun SHA-512 pada perangkat keras modern. BLAKE3 juga bersifat dapat diperluas (*extendable output*) sehingga panjang keluaran dapat disesuaikan dengan kebutuhan (Aumasson, 2021). Dalam penelitian yang dilakukan, penulis memilih BLAKE3 sebagai fungsi *fingerprinting* pada sistem CAS untuk menghasilkan identitas unik setiap *chunk*, memastikan performa *hashing* yang optimal pada operasi tulis dengan volume tinggi.

2.2.11 Aplikasi Web

Aplikasi web adalah perangkat lunak fungsional yang diakses melalui peramban web menggunakan infrastruktur jaringan internet atau intranet. Aplikasi web menerima masukan dari pengguna dan memproses instruksi logika pada sisi

server sebelum menyajikan hasil komputasi melalui antarmuka grafis yang dinamis. (Pressman, 2022). Dalam penelitian yang dilakukan, aplikasi web dikembangkan sebagai antarmuka klien layaknya dasbor penyimpanan awan untuk memfasilitasi pengguna dalam mengelola, mengunggah, dan mengunduh berkas cadangan secara interaktif.

2.2.12 Arsitektur Klien-Server

Arsitektur Klien-Server adalah model komputasi terdistribusi yang membagi beban operasional antara entitas penyedia sumber daya dan entitas peminta layanan. Klien memulai sesi komunikasi dengan mengirimkan paket permintaan, sementara server merespons dengan mengeksekusi logika bisnis dan mengembalikan data ke pihak peminta. (Kurose & Ross, 2021). Dalam penelitian yang dilakukan, arsitektur klien-server memisahkan peramban web sebagai klien pengirim muatan data, dengan *API Service* dan *Vault Core* sebagai server yang mengeksekusi logika pemecahan berkas dan penguncian fisik.

2.2.13 React

React adalah pustaka JavaScript sumber terbuka yang dirancang untuk membangun antarmuka pengguna secara deklaratif dan berbasis komponen. React mengelola status aplikasi secara efisien dengan memperbarui komponen visual secara spesifik saat terjadi perubahan data tanpa memicu proses pemuatan ulang pada keseluruhan halaman peramban. (Banks & Porcello, 2020). Dalam penelitian yang dilakukan, penulis mengimplementasikan pustaka React untuk membangun antarmuka halaman dasbor dan pratinjau berkas guna menyajikan status penguncian objek serta metrik deduplikasi secara seketika (*real-time*).

2.2.14 Basis Data

Basis data adalah kumpulan data terorganisasi yang disimpan secara sistematis dan dapat diakses, dikelola, serta diperbarui secara efisien. Model basis data diklasifikasikan berdasarkan cara pengorganisasian datanya, meliputi model relasional, dokumen, *key-value*, grafik, dan kolom lebar. Basis data *key-value* menyimpan data sebagai pasangan kunci-nilai dan menawarkan latensi baca-tulis yang sangat rendah, sehingga cocok untuk aplikasi yang memerlukan akses

metadata berskala tinggi (Kleppmann, 2024). Dalam penelitian yang dilakukan, sistem menggunakan basis data untuk menyimpan metadata *chunk* dan *manifest* objek yang diperlukan untuk operasi rekonstruksi berkas.

2.2.15 Log-Structured Merge-Tree (LSM-Tree)

LSM-Tree adalah struktur data yang dioptimalkan untuk performa tulis tinggi dengan cara menunda penulisan acak ke *disk* menjadi penulisan sekuensial bertahap. Struktur LSM-Tree terdiri atas komponen memori (*MemTable*) yang menyimpan data terbaru secara sementara, dan komponen *disk* (*Sorted String Table/SSTable*) yang menyimpan data secara permanen dalam format terurut. Proses *compaction* secara berkala menggabungkan level-level SSTable untuk mempertahankan performa baca yang stabil (Kleppmann, 2024). Dalam penelitian yang dilakukan, *LSM-Tree* menjadi fondasi mesin penyimpanan metadata karena operasi tulis pada sistem CAS bersifat intensif dan hampir tidak pernah memerlukan pembaruan di tempat (*in-place update*).

2.2.16 BadgerDB

BadgerDB adalah basis data *key-value* sumber terbuka yang diimplementasikan dalam Go dan dioptimalkan untuk performa tulis tinggi menggunakan struktur *LSM-Tree*. BadgerDB memisahkan penyimpanan kunci (*key*) dan nilai (*value*) ke berkas yang berbeda, strategi yang dikenal sebagai *key-value separation*, untuk mengurangi *write amplification* pada proses *compaction*. BadgerDB mendukung transaksi ACID dan operasi iterasi yang efisien (Kleppmann, 2024). Dalam penelitian yang dilakukan, sistem menggunakan BadgerDB untuk mengelola dua jenis metadata yaitu indeks keberadaan *chunk* berdasarkan nilai BLAKE3-nya dan *manifest* objek yang mencatat daftar *chunk* penyusun setiap berkas yang tersimpan.

2.2.17 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional tingkat lanjut yang menggunakan dan memperluas standar bahasa pemrograman SQL. Sistem mengelola relasi entitas terstruktur secara aman dan memastikan integritas data melalui implementasi prinsip ACID (*Atomicity, Consistency, Isolation, Durability*)

secara ketat pada setiap operasi penyisipan atau pembaruan. (Drake & Worsko, 2021). Dalam penelitian yang dilakukan, penulis mendedikasikan basis data PostgreSQL pada lingkungan publik (*Environment A*) untuk mengelola informasi dinamis seperti kredensial pengguna dan hierarki direktori secara terisolasi dari repositori utama *Vault*.

2.2.18 Autentikasi Pengguna

Autentikasi pengguna adalah proses pembuktian identitas entitas yang mencoba mengakses area terproteksi dari sebuah sistem komputasi. Sistem mencocokkan kredensial rahasia yang dikirimkan oleh pengguna dengan nilai kriptografi yang tersimpan pada basis data untuk menentukan keabsahan pemberian hak istimewa akses. (Stallings, 2023). Dalam penelitian yang dilakukan, mekanisme autentikasi diterapkan pada lapisan aplikasi web untuk membatasi akses dasbor hanya kepada pihak yang sah, sekaligus memvalidasi hak kepemilikan direktori sebelum sistem mengeksekusi perintah manajemen berkas.

2.2.19 Bahasa Pemrograman

Bahasa pemrograman adalah sistem notasi formal yang digunakan untuk mengekspresikan instruksi yang dapat dieksekusi oleh komputer. Bahasa pemrograman dikategorikan berdasarkan paradigmanya, antara lain prosedural, berorientasi objek, fungsional, dan konkurensi-pertama. Pemilihan bahasa pemrograman yang tepat berpengaruh signifikan terhadap performa, keandalan, dan kemudahan pemeliharaan sistem yang dibangun (Bodner, 2021). Dalam penelitian yang dilakukan, pemilihan bahasa pemrograman mempertimbangkan kemampuan penanganan konkurensi tinggi dan efisiensi operasi I/O asinkron yang diperlukan oleh sistem penyimpanan berperforma tinggi.

2.2.20 Go (Golang)

Go adalah bahasa pemrograman sumber terbuka yang dirancang oleh Google dengan fokus pada efisiensi eksekusi, kesederhanaan sintaksis, dan kemudahan dalam pemrograman konkurensi. Go menyediakan fitur *goroutine* sebagai unit konkurensi ringan yang dikelola oleh *runtime* Go, serta mekanisme *channel* untuk komunikasi antar *goroutine* secara aman. Go juga menyediakan

garbage collection, manajemen memori yang aman, serta pustaka standar yang kaya untuk pengembangan layanan jaringan dan sistem penyimpanan berperforma tinggi (Bodner, 2021). Penelitian yang dilakukan mengimplementasikan seluruh komponen sistem dalam Go, memanfaatkan *goroutine* untuk memproses operasi FastCDC dan penulisan *chunk* secara bersamaan, serta *Unix Domain Socket* (UDS) untuk komunikasi antara lapisan API dan mesin penyimpanan.

2.2.21 Keamanan Siber

Keamanan siber adalah praktik melindungi sistem komputer, jaringan, dan data dari serangan, kerusakan, atau akses tidak sah. Model pertahanan keamanan siber modern menganut prinsip pertahanan berlapis (*defense in depth*). Prinsip pertahanan berlapis merancang setiap lapisan sistem untuk tetap berfungsi meski lapisan lain telah dibobol. Konsep ini mencakup kontrol akses, enkripsi, pemantauan anomali, dan pemisahan hak istimewa (Stallings, 2023). Dalam penelitian yang dilakukan, arsitektur sistem menerapkan prinsip pertahanan berlapis dengan memisahkan domain kontrol aplikasi dari penyimpanan fisik, sehingga kompromi pada lapisan aplikasi tidak memberikan jalur untuk merusak data cadangan.

2.2.22 Ransomware

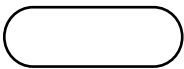
Ransomware adalah jenis perangkat lunak berbahaya (*malware*) yang mengenkripsi data korban menggunakan kunci yang hanya diketahui oleh penyerang, kemudian meminta pembayaran tebusan sebagai syarat pemulihan. Varian *ransomware* modern menggunakan strategi pemusnahan cadangan secara sistematis yaitu penyerang terlebih dahulu mengidentifikasi dan menghancurkan repositori *backup* sebelum mengenkripsi data utama, sehingga korban tidak memiliki pilihan pemulihan tanpa membayar tebusan. Serangan ini berhasil karena mayoritas arsitektur cadangan masih menggunakan jalur autentikasi yang sama antara aplikasi dan penyimpanan fisik, sehingga kredensial yang diretas memberikan akses penuh ke repositori cadangan (Stallings, 2023). Penelitian yang dilakukan secara langsung memitigasi vektor serangan ini dengan menerapkan

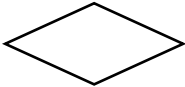
arsitektur yang secara struktural menolak operasi penghapusan dari jalur aplikasi manapun, termasuk dari akun dengan hak akses tertinggi.

2.2.23 Flowchart

Flowchart adalah penggambaran grafis dari langkah-langkah dan urutan prosedur dalam suatu program. *Flowchart* membantu analis dan pemrogram untuk memecahkan masalah menjadi segmen-segmen yang lebih kecil dan menganalisis alternatif dalam operasional. Dengan menggunakan simbol-simbol tertentu, *flowchart* menggambarkan urutan proses secara detail dan hubungan antara instruksi dalam suatu program. *Flowchart* juga mempermudah penyelesaian masalah, terutama yang memerlukan studi dan evaluasi lebih lanjut. Perangkat lunak pembelajaran *flowchart* dirancang untuk membantu pengguna memahami algoritma dan proses pembuatan *flowchart* tanpa perlu menulis kode program, serta menyediakan objek-objek yang diperlukan untuk membuat diagram alir. Berikut dilampirkan simbol-simbol *flowchart* yang tersedia pada Tabel 2.2 (Zalukhu dkk., 2023).

Tabel 2.2 Simbol Flowchart

Simbol	Nama	Fungsi
	Terminator	Permulaan/akhir program
	Garis alir (<i>flow line</i>)	Garis alir (<i>flow line</i>)
	<i>Preparation</i>	Proses inisialisasi/pemberian harga awal
	Proses	Proses perhitungan/proses pengolahan data
	Input/output data	Proses <i>input/output</i> data, parameter, informasi
	Sub program (<i>predefined process</i>)	Permulaan sub program/proses menjalankan sub program

Simbol	Nama	Fungsi
	<i>Decision</i>	Perbandingan pernyataan, penyeleksian data yang memberikan pilihan untuk langkah selanjutnya
	<i>On page connector</i>	Penghubung bagian-bagian <i>flowchart</i> yang berada pada satu halaman
	<i>Off page connector</i>	Penghubung bagian-bagian <i>flowchart</i> yang berada pada halaman berbeda

Sumber: Zalukhu (2023)

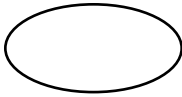
Tabel 2.2 menyajikan simbol-simbol standar yang digunakan dalam *flowchart* untuk mendeskripsikan alur dan proses sistem secara sistematis. Simbol "*Terminator*" digunakan untuk merepresentasikan awal atau akhir dari suatu proses, sedangkan "*Garis Alir (Flow Line)*" menunjukkan arah aliran proses dalam diagram. "*Preparation*" menggambarkan proses inisialisasi atau pemberian nilai awal, sementara "*Proses*" digunakan untuk mengilustrasikan tahapan pengolahan data. Simbol "*Input/Output Data*" menandakan aktivitas masukan atau keluaran data, dan "*Predefined Process*" merepresentasikan sub-program atau proses yang telah didefinisikan sebelumnya. "*Decision*" digunakan untuk proses pengambilan keputusan berdasarkan kondisi tertentu, dan "*On Page Connector*" serta "*Off Page Connector*" bertindak sebagai penghubung antar bagian *flowchart*, baik di halaman yang sama maupun halaman yang berbeda.

2.2.24 Data Flow Diagram

Data Flow Diagram (DFD) adalah alat yang digunakan untuk menggambarkan aliran data dalam suatu sistem yang berinteraksi dengan lingkungannya, membantu dalam mengidentifikasi kebutuhan pengguna dan merancang sistem dengan fokus pada struktur dan proses kerja. DFD, sering digunakan dalam pengembangan perangkat lunak berbasis metodologi seperti *System Development Life Cycle* (SDLC) dan *Structured System Analysis and Design Methodology* (SSADM). DFD memiliki komponen penting seperti entitas, proses, arus data, dan data store, yang berfungsi untuk memvisualisasikan

bagaimana data bergerak melalui sistem yang digambarkan dengan Tabel 2.3 (Ridwan dkk., 2022).

Tabel 2.3 Simbol Data Flow Diagram

Simbol	Nama
	External Entity
	Process
	Data Storage
	Data Flow

Sumber: Ridwan (2022)

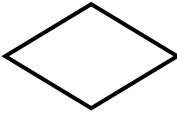
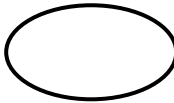
Tabel 2.3 menggambarkan simbol-simbol standar yang digunakan dalam *Data Flow Diagram* (DFD) untuk merepresentasikan elemen-elemen utama dalam analisis sistem. Simbol "*External Entity*" berbentuk oval digunakan untuk merepresentasikan entitas eksternal yang berinteraksi dengan sistem, seperti pengguna atau organisasi lain. Simbol "*Process*" berbentuk persegi panjang menunjukkan aktivitas atau fungsi dalam sistem yang mengolah data untuk menghasilkan *output*. "*Data Storage*" direpresentasikan dengan simbol dua garis sejajar, yang menunjukkan tempat penyimpanan data dalam sistem, baik secara sementara maupun permanen. Terakhir, simbol "*Data Flow*" berupa panah menunjukkan arah aliran data antara entitas, proses, dan penyimpanan.

2.2.25 Entity Relationship Diagram

Entity Relationship Diagram (ERD) adalah teknik pemodelan basis data untuk menggambarkan hubungan antar entitas dalam suatu sistem. ERD berfungsi sebagai representasi grafis dari model data konseptual yang mencerminkan kebutuhan data pengguna dalam sistem basis data. Dalam ERD, terdapat tiga elemen dasar: entitas, atribut, dan relasi. Entitas adalah objek yang dapat berupa manusia, tempat, atau benda, yang diwakili oleh simbol persegi panjang. ERD

merupakan tahap awal dalam desain basis data dan penting untuk memastikan bahwa semua entitas saling terhubung dengan benar, serta untuk menghindari kesalahan konseptual, prosedural, dan teknis dalam pembuatan basis data. Berikut dilampirkan simbol-simbol ERD yang tersedia pada Tabel 2.4 (Pulungan dkk., 2022).

Tabel 2.4 Simbol Entity Relationship Diagram

Nama	Gambar	Keterangan
Entitas		Entitas, suatu objek yang dapat dijadikan didefinisikan dalam lingkungan pemakai.
Relasi		Relasi, menunjukkan adanya hubungan diantara sejumlah entitas yang berbeda.
Atribut		Atribut, mendeskripsikan karakter entitas (atribut yang berfungsi sebagai kunci yang diberi garis bawah).
Garis		Garis, penghubung antara relasi dengan entitas, relasi dan entitas dengan atribut.

Sumber: Pulungan (2022)

Tabel 2.4 menjelaskan simbol-simbol dasar dalam *Entity Relationship Diagram* (ERD) yang digunakan untuk memodelkan struktur dan hubungan data dalam sistem basis data. Simbol "Entitas," yang direpresentasikan dengan persegi panjang, merepresentasikan objek yang dapat didefinisikan, seperti "Pengguna" atau "Produk." Simbol "Relasi," berbentuk wajik, menggambarkan hubungan antara dua atau lebih entitas, misalnya relasi "Memesan" antara entitas "Pelanggan" dan "Produk." Atribut entitas atau relasi direpresentasikan dengan oval, yang mendeskripsikan karakteristik atau properti entitas. Atribut kunci unik diberi garis bawah yang berfungsi sebagai penghubung untuk menunjukkan keterkaitan antara entitas, relasi, dan atribut.

2.2.26 Pengujian Black Box

Pengujian *black box* adalah metode pengujian perangkat lunak yang mengevaluasi fungsionalitas sistem berdasarkan spesifikasi keluaran yang diharapkan, tanpa memperhatikan struktur atau logika internal kode program. Penguji berinteraksi dengan sistem semata-mata melalui antarmuka yang tersedia, kemudian membandingkan keluaran aktual dengan keluaran yang diharapkan sesuai spesifikasi. Pengujian *black box* efektif mendeteksi ketidaksesuaian fungsi, kesalahan antarmuka, dan perilaku yang menyimpang dari spesifikasi tanpa bergantung pada pengetahuan implementasi (Pressman & Maxim, 2020). Dalam penelitian yang dilakukan, penulis menerapkan pengujian *black box* pada seluruh endpoint API sistem *Immutable Object Storage* untuk memverifikasi bahwa setiap operasi unggah, unduh, dan pengelolaan retensi menghasilkan respons yang sesuai spesifikasi, termasuk penanganan masukan tidak valid dan kondisi batas ukuran objek.

2.2.27 Pengujian Keamanan

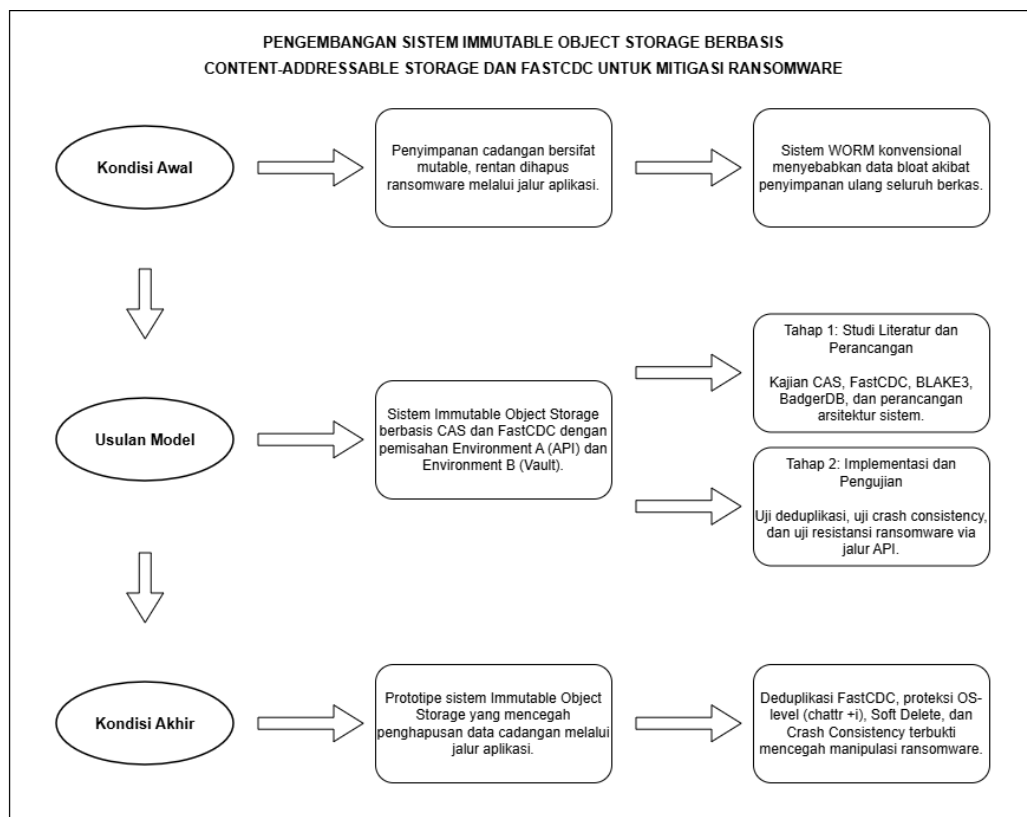
Pengujian keamanan (*security testing*) adalah proses evaluasi sistematis yang bertujuan memverifikasi bahwa mekanisme pertahanan suatu sistem bekerja sesuai klaim dan mengidentifikasi kerentanan sebelum sistem dieksploitasi secara nyata. Pengujian keamanan mencakup simulasi serangan terhadap lapisan autentikasi, otorisasi, isolasi hak akses, dan ketahanan terhadap manipulasi data, dengan mengacu pada kerangka metodologis seperti *OWASP Testing Guide* dan *NIST SP 800-115* (Stallings, 2023). Dalam penelitian yang dilakukan, penulis merancang pengujian keamanan untuk membuktikan klaim pertahanan utama sistem terhadap serangan *ransomware* dengan mensimulasikan kondisi pasca-peretasan, di mana penyerang yang telah menguasai lapisan API mencoba menjalankan operasi penghapusan massal dan penyimpanan objek melalui jalur *Unix Domain Socket* (UDS) untuk membuktikan bahwa arsitektur pemisahan domain secara struktural menolak seluruh operasi destruktif tersebut.

BAB III

METODE PENELITIAN

3.1 Kerangka Penelitian

Penulis menyusun kerangka penelitian untuk merumuskan tahapan pengembangan sistem secara terstruktur. Penulis mengawali kerangka penelitian dengan menetapkan kondisi awal melalui proses identifikasi kerentanan repositori cadangan terhadap serangan *ransomware*. Setelah merumuskan akar permasalahan, penulis mengumpulkan data teknis dan merancang arsitektur sistem penyimpanan hibrida sebagai solusi perlindungan data. Tahap penyelesaian penelitian berfokus pada pencapaian kondisi ideal melalui eksekusi pengujian implementasi guna memvalidasi tingkat keamanan dan fungsionalitas prototipe. Gambar 3.1 menunjukkan alur kerangka penelitian.



Gambar 3.1 Kerangka Penelitian

Gambar 3.1 menunjukkan kerangka penelitian. Penjabaran tahapan kerangka penelitian diuraikan secara berurutan dari awal hingga akhir. Tahap kondisi awal mencakup langkah penulis dalam mengidentifikasi masalah dengan

menganalisis kerentanan arsitektur penyimpanan cadangan terhadap serangan *ransomware*. Langkah selanjutnya adalah tahap pengumpulan data primer dan sekunder untuk menentukan spesifikasi kebutuhan sistem. Berdasarkan data yang terkumpul, penulis melakukan perancangan arsitektur *Immutable Object Storage* yang mengintegrasikan mekanisme *Content-Addressable Storage* (CAS) dan algoritma FastCDC. Penulis kemudian mengimplementasikan rancangan sistem ke dalam bentuk prototipe perangkat lunak. Setelah tahap implementasi selesai, penulis menguji sistem menggunakan pengujian performa dan simulasi serangan keamanan. Tahap kondisi akhir menghasilkan solusi berupa sistem penyimpanan data yang teruji mampu memitigasi modifikasi paksa dari *ransomware* secara struktural.

3.2 Data Penelitian

3.2.1 Sumber Data

Penulis menggunakan dua klasifikasi sumber data, yakni data primer dan data sekunder. Penulis mengumpulkan data primer secara langsung dari objek penelitian, sedangkan pihak lain memublikasikan data sekunder sebagai referensi teknis lanjutan. Sebagai data primer, penulis menggunakan kumpulan sampel berkas biner, dokumen teks, dan gambar untuk menguji performa pemecahan data (*chunking*). Tabel 3.1 menyajikan rincian sumber data penelitian.

Tabel 3.1 Sumber Data Penelitian

Jenis Data	Kategori	Deskripsi	Sumber
Primer	Sampel Uji Coba	Kumpulan sampel berkas biner, dokumen teks, dan gambar dengan berbagai ukuran untuk pengujian pemecahan data (<i>chunking</i>) dan performa <i>throughput</i> .	Simulasi mandiri oleh penulis pada lingkungan komputasi lokal.

3.2.2 Mendapatkan Data

Penulis memperoleh data primer melalui pembuatan sampel berkas simulasi secara mandiri (*dummy files*) pada lingkungan komputasi lokal. Penulis merancang sampel berkas dengan berbagai tingkat entropi untuk merepresentasikan

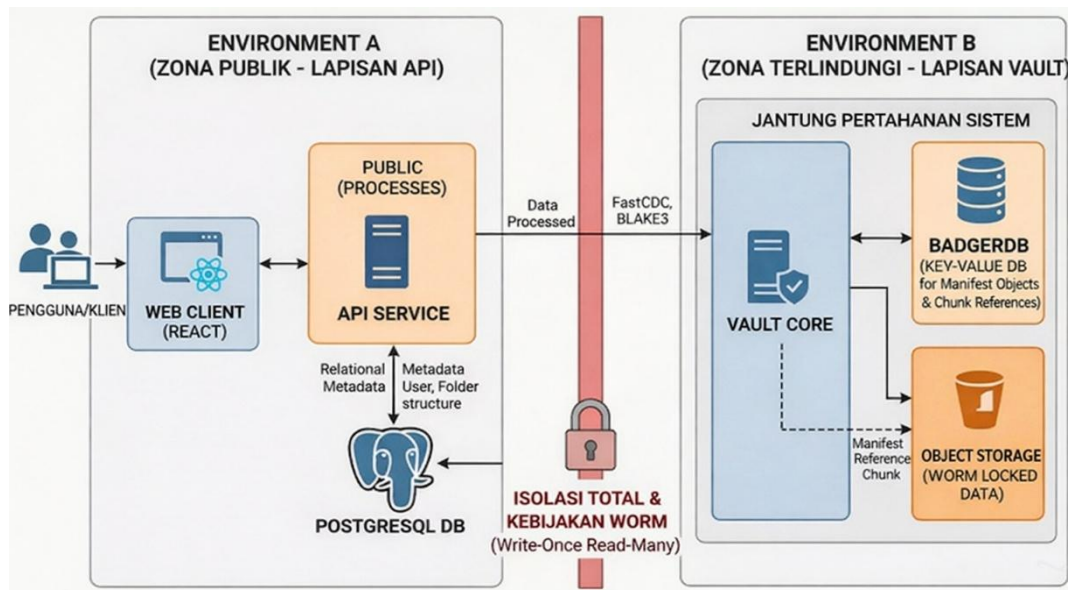
karakteristik dokumen cadangan pada skenario dunia nyata. Sistem memproses sampel berkas simulasi tersebut guna menguji tingkat efisiensi pemecahan algoritma FastCDC. Selanjutnya, penulis memperoleh data sekunder melalui studi pustaka dengan mengunduh dokumentasi teknis resmi. Penulis juga mengumpulkan artikel jurnal akademik melalui pangkalan data daring menggunakan kata kunci spesifik terkait penyimpanan *immutable* dan mitigasi *ransomware*.

3.2.3 Waktu Pengumpulan Data

Penulis melaksanakan proses pengumpulan data selama empat minggu, terhitung sejak minggu pertama hingga minggu keempat masa pelaksanaan Proyek Utama Informatika. Penulis mengalokasikan jadwal pengumpulan data secara paralel dengan tahapan analisis masalah dan perumusan studi literatur. Strategi penjadwalan paralel bertujuan memastikan penulis menguasai seluruh spesifikasi teknis dan algoritma pendukung sebelum memulai tahap implementasi kode program.

3.3 Arsitektur Model

Arsitektur model menggambarkan struktur hibrida yang memisahkan lingkungan publik dari lingkungan penyimpanan. Penulis merancang sistem dengan membagi domain kontrol ke dalam dua zona isolasi guna memitigasi risiko modifikasi data oleh *ransomware*. Gambar 3.2 menunjukkan arsitektur model *Immutable Object Storage*.



Gambar 3.2 Arsitektur Model

Environment A beroperasi sebagai zona publik yang menangani seluruh interaksi pengguna. Lingkungan publik terdiri dari tiga komponen utama, yaitu *Web Client*, *API Service*, dan basis data relasional PostgreSQL. *Web Client* yang dibangun menggunakan React menyajikan antarmuka grafis untuk manajemen berkas dan direktori. *API Service* menerima permintaan dari klien, mengeksekusi logika autentikasi, serta mengelola metadata relasional pada basis data PostgreSQL. Basis data PostgreSQL menyimpan informasi dinamis seperti identitas pengguna, struktur folder, dan pemetaan kepemilikan berkas.

Environment B beroperasi sebagai *Vault Core* yang sangat terisolasi untuk mengelola penyimpanan fisik dan metadata permanen. Saat proses unggah berlangsung, *API Service* memecah aliran data menggunakan algoritma FastCDC dan menghasilkan nilai *hash* BLAKE3. Selanjutnya, *API Service* mengirimkan hasil pemrosesan data menuju *Vault Core* agar sistem dapat menguncinya secara permanen. *Vault Core* menggunakan basis data *key-value* BadgerDB untuk mencatat manifes objek dan referensi *chunk*. Integrasi antara kebijakan *Write-Once Read-Many* (WORM) pada *Vault Core* dan isolasi basis data menjamin integritas data tetap terjaga meskipun lapisan aplikasi pada *Environment A* mengalami kompromi keamanan.

3.4 Analisis dan Perancangan

3.4.1 Kebutuhan Fungsional

Kebutuhan fungsional menjelaskan interaksi antara pengguna dan sistem, yang dikelompokkan menjadi tiga bagian utama, yakni kebutuhan masukan, kebutuhan proses, dan kebutuhan luaran.

a. Kebutuhan Masukan

Kebutuhan masukan mendefinisikan seluruh data atau instruksi yang dikirimkan oleh klien ke dalam sistem. Kebutuhan masukan meliputi:

- 1) Pengguna memasukkan kredensial autentikasi berupa alamat surel dan kata sandi untuk mengakses antarmuka web.
- 2) Pengguna memasukkan instruksi pembuatan direktori baru beserta nama folder yang diinginkan.
- 3) Pengguna mengirimkan aliran data berkas cadangan dengan berbagai ukuran dan ekstensi melalui halaman dasbor.
- 4) Pengguna mengirimkan parameter aksi manajemen berkas, seperti perintah unduh atau perintah hapus (*soft delete*).

b. Kebutuhan Proses

Sistem menjalankan serangkaian proses komputasi terhadap data masukan yang diterima. Daftar kebutuhan proses mencakup:

- 1) Sistem memvalidasi kredensial pengguna dan mengelola sesi autentikasi secara aman.
- 2) Sistem mencatat informasi kepemilikan berkas, riwayat aktivitas, dan hierarki direktori ke dalam basis data relasional PostgreSQL pada *Environment A*.
- 3) Sistem memecah aliran data berkas menjadi potongan (*chunk*) berukuran dinamis menggunakan algoritma FastCDC.
- 4) Sistem menghitung nilai *hash* BLAKE3 untuk setiap potongan data guna mengeksekusi deduplikasi secara otomatis.

- 5) Sistem mengunci data fisik dan menyimpan manifes objek secara permanen ke dalam basis data BadgerDB pada *Environment B*.
- 6) Sistem memproses instruksi penghapusan berkas hanya pada lapisan antarmuka pengguna tanpa menghapus fisik objek di dalam *Vault*

c. Kebutuhan Luaran

Kebutuhan luaran mendefinisikan hasil pemrosesan yang dikembalikan oleh sistem kepada pengguna. Kebutuhan luaran meliputi:

- 1) Sistem menyajikan antarmuka grafis yang memuat daftar direktori, metrik kapasitas, dan riwayat aktivitas pengguna.
- 2) Sistem menampilkan panel pratinjau keamanan yang memuat informasi status penguncian objek (*immutable*) dan nilai *hash* kriptografi.
- 3) Sistem merespons permintaan unduh dengan mengembalikan struktur berkas yang utuh ke penyimpanan lokal pengguna.
- 4) Sistem memberikan umpan balik visual (*notifikasi*) mengenai status keberhasilan atau kegagalan setiap operasi sistem.

3.4.2 Kebutuhan Non Fungsional

Kebutuhan non fungsional mencakup spesifikasi lingkungan kerja yang mendukung jalannya aplikasi secara optimal.

a. Kebutuhan Perangkat Lunak

Penulis menggunakan beberapa perangkat lunak untuk mengembangkan dan menjalankan sistem:

- 1) Sistem Operasi: Windows 11 dengan Windows Subsystem for Linux (WSL2).
- 2) Bahasa Pemrograman: Go versi 1.21 dan NodeJS versi 25.9.0.
- 3) Kerangka Kerja Aplikasi Web: ReactJS
- 4) Basis Data: BadgerDB dan PostgreSQL.
- 5) Alat Pengujian: Postman API Platform dan Chrome Browser.

b. Kebutuhan Perangkat Keras

Sistem membutuhkan spesifikasi perangkat keras minimum untuk menjamin kelancaran komputasi kriptografi:

- 1) Prosesor (CPU): Prosesor quad-core (contoh: Intel Core i5 atau AMD Ryzen 5).
- 2) Memori (RAM): Minimal 8 Gigabita (GB) untuk mendukung konkurensi data memori.
- 3) Media Penyimpanan: Solid State Drive (SSD) dengan kapasitas ruang minimal 50 GB guna mendukung kecepatan operasi baca-tulis BadgerDB.

3.4.3 Perancangan Konseptual dan Fisik

Penulis menyusun rancangan konseptual untuk memvisualisasikan alur informasi sistem dan rancangan fisik untuk mendokumentasikan struktur penyimpanan data. Rancangan konseptual mencakup *Entity Relationship Diagram* (ERD), *Data Flow Diagram* (DFD) dari Level 0 hingga Level 2, serta *wireframe* antarmuka *endpoint* API. Rancangan fisik mendokumentasikan skema struktur data BadgerDB yang menjadi fondasi sistem penyimpanan.

3.4.3.1 Perancangan Konseptual

a. Entity Relationship Diagram (ERD)

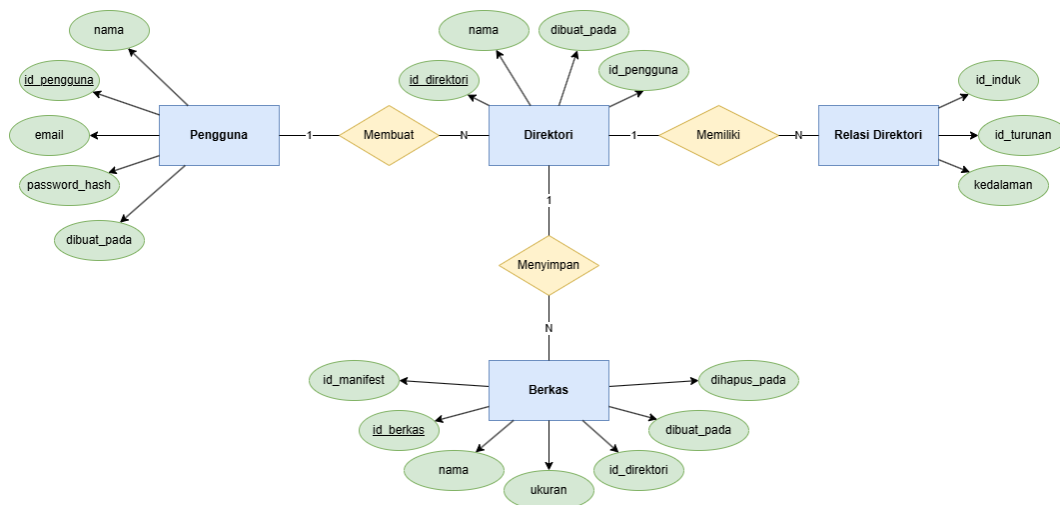
Gambar 3.3 menunjukkan *Entity Relationship Diagram* (ERD) yang merepresentasikan skema basis data relasional untuk sistem penyimpanan berkas. Penulis merancang struktur data yang terdiri dari empat entitas utama, yaitu entitas pengguna, entitas direktori, entitas relasi direktori, dan entitas berkas. Penulis mengimplementasikan pendekatan *Closure Table* untuk mendukung struktur sub-direktori dengan kedalaman tak terbatas tanpa menggunakan relasi rekursif pada entitas direktori.

Entitas pengguna menyimpan informasi kredensial dan identitas pemilik data. Entitas pengguna memiliki relasi *one-to-many* terhadap entitas direktori, yang berarti satu pengguna dapat memiliki banyak direktori. Entitas direktori berfungsi sebagai representasi folder fisik tanpa menyimpan referensi langsung ke folder induk di dalam tabel yang sama. Strategi ini memisahkan logika organisasi folder dari logika struktur hierarki.

Entitas relasi direktori menyimpan pemetaan hierarki folder secara eksplisit. Entitas relasi direktori menghubungkan dua identitas dari entitas direktori, yaitu identitas induk (*id_induk*) dan identitas turunan (*id_turunan*). Dengan menyimpan

seluruh jalur hubungan antar direktori, sistem dapat melakukan pencarian seluruh isi sub-direktori dengan performa yang optimal tanpa melakukan kueri berulang.

Entitas berkas menyimpan metadata dokumen yang diunggah oleh pengguna ke dalam sistem. Entitas berkas memiliki relasi one-to-many dengan entitas direktori. Setiap berkas terikat pada satu direktori spesifik sebagai lokasi penyimpanan logis di dalam arsitektur sistem.



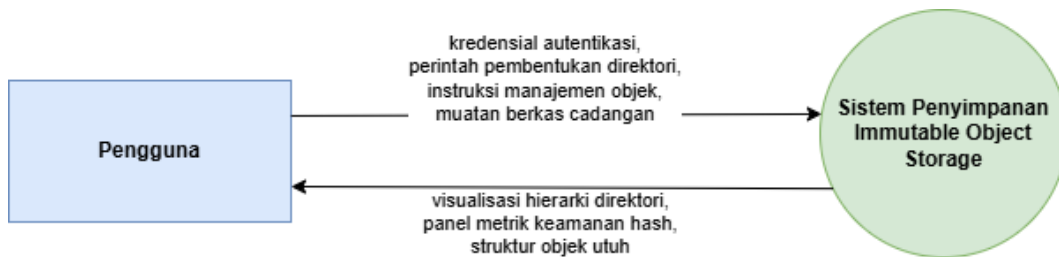
Gambar 3. 3 Entity Relationship Diagram (ERD)

b. Data Flow Diagram (DFD)

Penulis merancang *Data Flow Diagram* (DFD) untuk memetakan alur data pada sistem *Immutable Object Storage* secara bertingkat dari Level 0 hingga Level 2. DFD Level 0 menggambarkan interaksi sistem secara keseluruhan, DFD Level 1 mengurai proses-proses utama, dan DFD Level 2 merinci setiap sub-proses secara lebih mendalam.

1) DFD Level 0 (Context Diagram)

Gambar 3.4 menunjukkan Data Flow Diagram (DFD) Level 0 atau Diagram Konteks. Entitas eksternal pengguna berinteraksi secara dua arah dengan sistem penyimpanan *Immutable Object Storage*. Pengguna mengirimkan aliran data masukan berupa kredensial autentikasi, perintah pembentukan direktori, instruksi manajemen objek, dan muatan berkas cadangan. Sistem merespons aliran data masukan dengan mengembalikan visualisasi hierarki direktori pada dasbor web, panel metrik keamanan *hash*, serta struktur objek utuh saat operasi unduhan berlangsung.



Gambar 3.4 Data Flow Diagram (DFD) Level 0

2) DFD Level 1

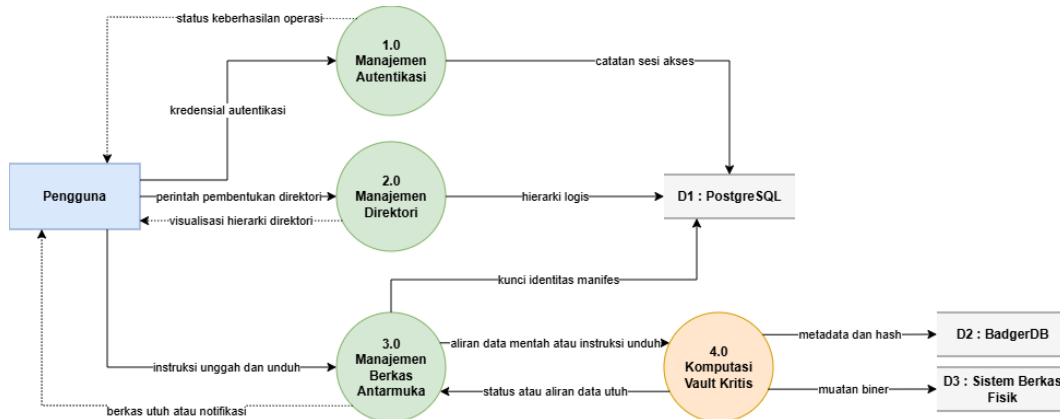
Gambar 3.5 menunjukkan *Data Flow Diagram* (DFD) Level 1 yang menjabarkan dekomposisi proses komputasi utama sistem. Sistem mendistribusikan aliran data ke dalam empat proses fungsional inti, yakni proses manajemen autentikasi (1.0), proses manajemen direktori (2.0), proses manajemen berkas antarmuka (3.0), dan proses komputasi *Vault* kritis (4.0).

Proses 1.0 Manajemen Autentikasi memvalidasi identitas pengguna dan mencatat sesi akses ke dalam penyimpanan data PostgreSQL. Proses 2.0 Manajemen Direktori menerima perintah pembentukan tata letak folder dan menyimpan hierarki logis ke dalam penyimpanan data PostgreSQL.

Proses 3.0 Manajemen Berkas Antarmuka bertindak sebagai gerbang operasional utama yang menerima instruksi unggah dan instruksi unduh dari pengguna. Pada mekanisme pengunggahan, proses 3.0 Manajemen Berkas Antarmuka meneruskan aliran data mentah menuju proses 4.0 Komputasi *Vault* Kritis yang berada di dalam lingkungan terisolasi. Proses 4.0 Komputasi *Vault* Kritis mengeksekusi algoritma FastCDC untuk memecah data dan algoritma BLAKE3 untuk menghasilkan nilai kriptografi. Selanjutnya, proses 4.0 Komputasi *Vault* Kritis menyimpan muatan biner fisik ke dalam entitas Sistem Berkas Fisik, serta mengunci metadata permanen ke dalam penyimpanan data BadgerDB.

Sebagai tahap akhir sinkronisasi hibrida pada proses unggah, proses 4.0 Komputasi *Vault* Kritis mengembalikan kunci identitas manifes kepada proses 3.0 Manajemen Berkas Antarmuka. Proses 3.0 Manajemen Berkas Antarmuka menyimpan kunci identitas manifes ke dalam PostgreSQL sebagai jembatan penunjuk referensi data. Sebaliknya pada mekanisme pengunduhan, proses 4.0 Komputasi *Vault* Kritis melakukan rekonstruksi objek dengan mengambil muatan

fisik dari Sistem Berkas Fisik, lalu mengembalikan aliran data utuh kepada proses 3.0 Manajemen Berkas Antarmuka untuk disajikan kembali kepada pengguna.

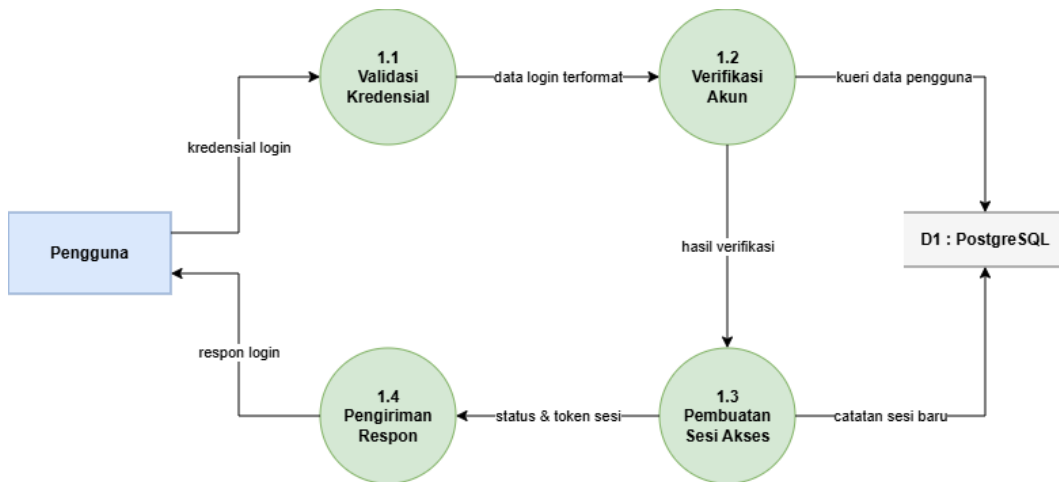


Gambar 3.5 Data Flow Diagram (DFD) Level 1

3) DFD Level 2

a) DFD Level 2 Proses 1 (Manajemen Autentikasi)

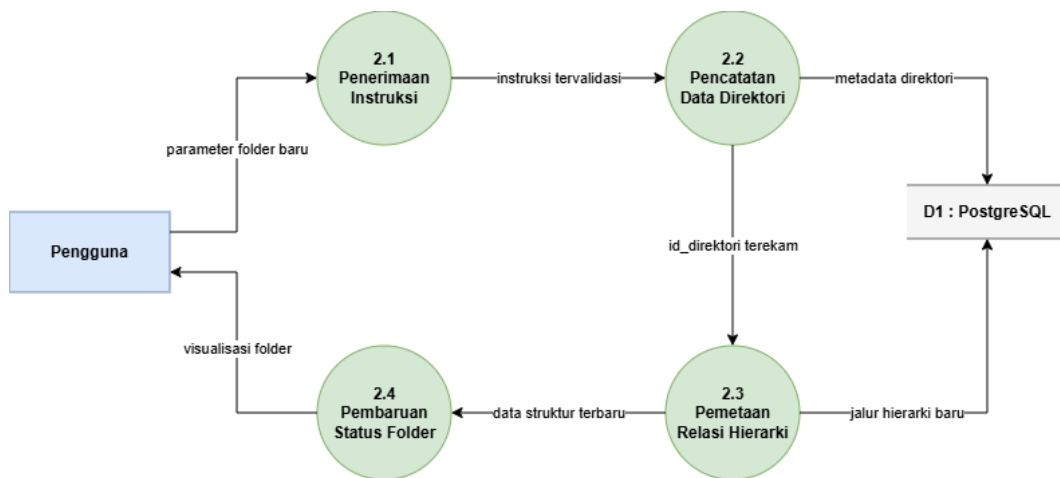
Gambar 3.6 menunjukkan DFD Level 2 Proses 1 yang menguraikan rincian alur kerja pada bagian Manajemen Autentikasi. Penulis membagi Proses 1 ke dalam empat sub-proses utama guna menjamin keamanan akses pengguna sebelum melakukan interaksi dengan sistem penyimpanan. Tahapan autentikasi melibatkan verifikasi identitas serta pengelolaan sesi aktif pengguna secara sistematis. Sub-proses 1.1 Validasi Kredensial menerima masukan berupa nama pengguna dan kata sandi dari entitas Pengguna. Selanjutnya, sub-proses 1.2 Verifikasi Akun melakukan kueri ke dalam tabel pengguna pada basis data PostgreSQL untuk mencocokkan nilai *hash* kata sandi. Sub-proses 1.3 Pembuatan Sesi Akses membentuk token akses unik setelah sistem berhasil memvalidasi identitas pengguna. Tahap akhir dilakukan oleh sub-proses 1.4 Pengiriman Respon yang meneruskan status keberhasilan login dan token sesi kembali kepada Pengguna sebagai otoritas untuk mengakses fitur sistem lainnya.



Gambar 3.6 Data Flow Diagram (DFD) Level 2 Proses 1

b) DFD Level 2 Proses 2 (Manajemen Direktori)

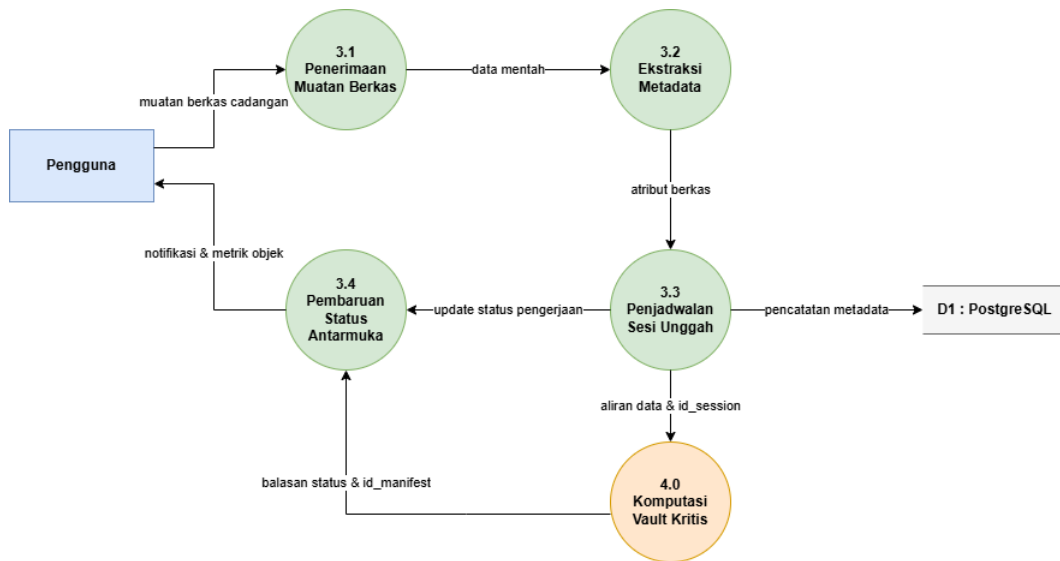
Gambar 3.7 menunjukkan DFD Level 2 Proses 2 yang menguraikan rincian mekanisme pengelolaan struktur folder di dalam sistem. Penulis merancang empat sub-proses utama untuk menangani pembentukan direktori serta pemetaan hierarki menggunakan pendekatan *Closure Table*. Sub-proses 2.1 Penerimaan Instruksi Direktori menerima parameter nama folder dan identitas folder induk dari entitas Pengguna. Selanjutnya, sub-proses 2.2 Pencatatan Data Direktori menyimpan metadata dasar folder ke dalam tabel direktori pada basis data PostgreSQL. Sub-proses 2.3 Pemetaan Relasi Hierarki menjalankan logika untuk mendaftarkan hubungan antara folder baru dengan seluruh leluhurnya di dalam tabel relasi direktori guna mendukung struktur bersarang tanpa batas kedalaman. Tahap akhir dilakukan oleh sub-proses 2.4 Pembaruan Status Folder yang mengirimkan konfirmasi keberhasilan serta data struktur terbaru kembali kepada Pengguna untuk memperbarui tampilan antarmuka.



Gambar 3.7 Data Flow Diagram (DFD) Level 2 Proses 2

c) DFD Level 2 Proses 3 (Manajemen Berkas Antarmuka)

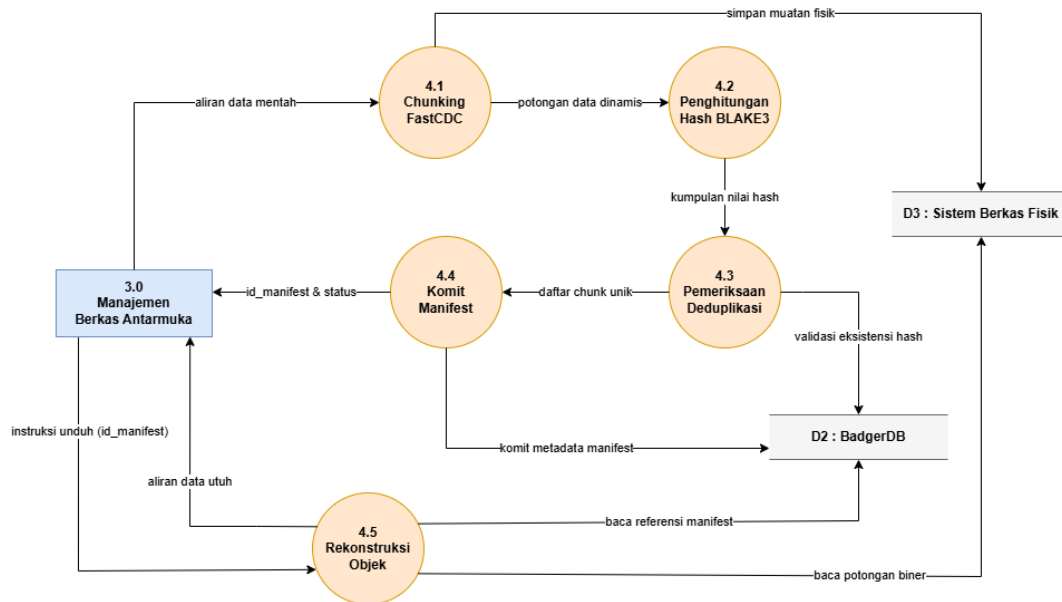
Gambar 3.8 menunjukkan DFD Level 2 Proses 3 yang menguraikan rincian mekanisme manajemen berkas pada lapisan antarmuka sistem. Penulis merancang empat sub-proses utama untuk menjembatani interaksi antara entitas Pengguna dengan lapisan komputasi kritis di dalam lingkungan *Vault Core*. Sub-proses 3.1 Penerimaan Muatan Berkas menerima aliran data berkas cadangan dari Pengguna melalui protokol HTTP. Selanjutnya, sub-proses 3.2 Ekstraksi Metadata melakukan identifikasi awal terhadap karakteristik berkas seperti nama, ukuran, dan tipe berkas. Sub-proses 3.3 Penjadwalan Sesi Unggah membentuk identitas sesi sementara dan mendaftarkan metadata awal ke dalam tabel berkas pada basis data PostgreSQL. Tahap akhir dilakukan oleh sub-proses 3.4 Pembaruan Status Antarmuka yang mengirimkan notifikasi progres serta hasil identifikasi objek kembali kepada Pengguna setelah menerima umpan balik dari Proses 4.0.



Gambar 3.8 Data Flow Diagram (DFD) Level 2 Proses 3

d) DFD Level 2 Proses 4 (Manajemen Vault Kritis)

Gambar 3.9 menunjukkan DFD Level 2 Proses 4 yang menguraikan rincian mekanisme komputasi kritis di dalam lingkungan *Vault Core*. Penulis merancang lima sub-proses utama untuk menjamin integritas data dan efisiensi penyimpanan. Sub-proses 4.1 *Chunking* FastCDC menerima aliran data mentah dan memotongnya menjadi bagian biner dinamis yang kemudian disimpan langsung ke dalam D3: Sistem Berkas Fisik. Sub-proses 4.2 Penghitungan *Hash* BLAKE3 menentukan identitas unik setiap potongan data, sementara sub-proses 4.3 Pemeriksaan Deduplikasi memvalidasi eksistensi *hash* pada D2: BadgerDB guna menghindari penyimpanan data ganda. Tahap penyimpanan akhir dilakukan oleh sub-proses 4.4 Komit Manifes yang mengunci metadata dokumen secara permanen. Selain alur pengunggahan, penulis menambahkan sub-proses 4.5 Rekonstruksi Objek untuk menangani permintaan unduh. Proses ini membaca referensi manifes dari BadgerDB, mengambil potongan biner dari Sistem Berkas Fisik, dan merakitnya kembali menjadi aliran data utuh untuk dikirimkan kembali ke antarmuka pengguna.

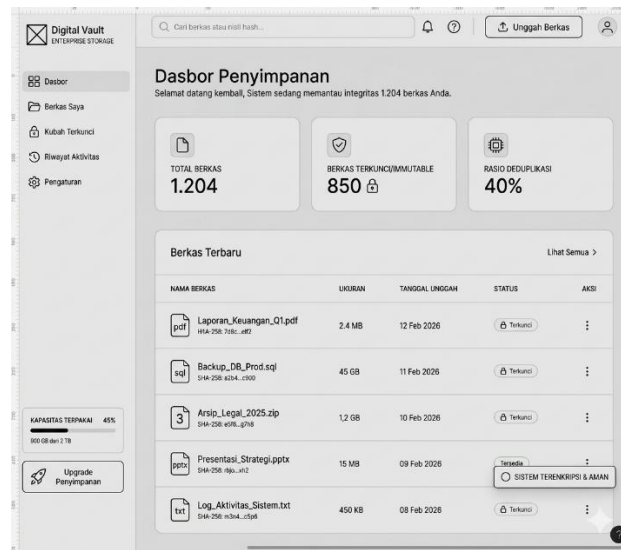


Gambar 3.9 Data Flow Diagram (DFD) Level 2 Proses 4

c. Wireframe

Sistem *Immutable Object Storage* menyediakan antarmuka grafis berbasis web untuk memfasilitasi interaksi klien. Penulis menyusun rancangan antarmuka (*wireframe*) guna memvisualisasikan tata letak komponen visual pada halaman aplikasi pengguna. Rancangan visual memuat halaman dasbor utama sebagai pusat kendali manajemen direktori, serta halaman pratinjau berkas yang menyajikan informasi kriptografi dan status penguncian objek secara terperinci.

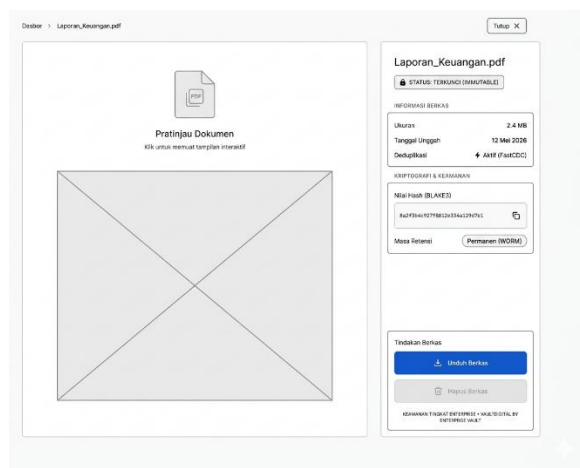
1) Wireframe Halaman Dasbor



Gambar 3.10 Wireframe Halaman Dasbor

Gambar 3.10 menunjukkan *wireframe* antarmuka halaman dasbor utama berbasis web. Halaman dasbor berfungsi sebagai ruang kerja bagi pengguna untuk mengelola dokumen cadangan. Sistem menampilkan daftar berkas yang telah diunggah beserta informasi metadata dasar seperti nama berkas, ukuran, dan tanggal unggah. Pengguna mengunggah berkas baru melalui tombol aksi utama yang memicu jendela pemilihan berkas. Antarmuka web mengirimkan aliran data menuju lapisan API untuk melalui proses pemecahan oleh mesin FastCDC.

2) Wireframe Halaman Pratinjau Berkas



Gambar 3.11 Wireframe Halaman Pratinjau Berkas

Gambar 3.11 menunjukkan *wireframe* antarmuka halaman rincian berkas. Halaman rincian berkas menyajikan informasi spesifik mengenai objek yang

tersimpan, termasuk nilai *hash* kriptografi BLAKE3 dan status retensi dari *Vault Core*. Sistem memvisualisasikan indikator penguncian objek secara grafis guna memberikan jaminan bahwa berkas berstatus *immutable* dan aman dari serangan *ransomware*. Sistem juga menyediakan tombol unduh bagi pengguna untuk mengambil kembali berkas cadangan secara utuh ke penyimpanan lokal.

3.4.3.2 Perancangan Fisik

Perancangan fisik basis data mendokumentasikan spesifikasi teknis dari setiap tabel yang digunakan dalam pengembangan sistem. Penulis menggunakan sistem manajemen basis data PostgreSQL untuk menjamin integritas data dan efisiensi penyimpanan.

a. Basis Data Relasional

1) Tabel Pengguna

Tabel 3.2 menyajikan struktur entitas pengguna. Entitas pengguna menyimpan kredensial autentikasi klien yang mengakses aplikasi web.

Tabel 3.2 Tabel Pengguna

No	Nama Kolom	Tipe Data	Keterangan
1	id_pengguna	UUID	<i>Primary Key</i> , identitas unik pengguna.
2	nama	VARCHAR	Nama lengkap pengguna aplikasi.
3	email	VARCHAR	Alamat surel (<i>email</i>) pengguna untuk keperluan <i>login</i> .
4	password_hash	VARCHAR	Kata sandi pengguna yang telah dienkripsi (<i>di-hash</i>).
5	dibuat_pada	TIMESTAMP	Waktu pembuatan akun pengguna.

2) Tabel Direktori

Tabel 3.3 menyajikan struktur entitas berkas pada lapisan aplikasi. Entitas berkas klien berfungsi sebagai jembatan yang menghubungkan kepemilikan pengguna di antarmuka web dengan identitas objek di dalam *Vault*.

Tabel 3.3 Tabel Direktori

No	Nama Kolom	Tipe Data	Keterangan
1	id_direktori	UUID	<i>Primary Key</i> , identitas unik direktori.

No	Nama Kolom	Tipe Data	Keterangan
2	id_pengguna	UUID	<i>Foreign Key</i> , merujuk pada identitas pemilik di tabel Pengguna.
3	nama	VARCHAR	Nama direktori yang dibuat oleh pengguna.
4	dibuat_pada	TIMESTAMP	Waktu pembuatan direktori.

3) Tabel Relasi Direktori

Tabel 3.4 menyajikan struktur entitas relasi direktori. Tabel relasi direktori mengimplementasikan logika tabel penutup dengan menggunakan id_induk sebagai referensi induk dalam struktur hierarki.

No	Nama Kolom	Tipe Data	Keterangan
1	id_induk	UUID	Primary Key & Foreign Key, identitas direktori tingkat atas.
2	id_turunan	UUID	Primary Key & Foreign Key, identitas direktori tingkat bawah.
3	kedalaman	INTEGER	Jarak tingkat antara induk dan turunan.

4) Tabel Berkas

Tabel 3.5 menyajikan struktur entitas berkas. Tabel berkas menyimpan seluruh informasi dokumen yang dikelola oleh sistem di bawah direktori tertentu.

Tabel 3. 4 Tabel Berkas

No	Nama Kolom	Tipe Data	Keterangan
1	id_berkas	UUID	<i>Primary Key</i> , identitas unik data berkas pada aplikasi web.
2	id_direktori	UUID	<i>Foreign Key</i> , merujuk pada lokasi penyimpanan berkas di tabel Direktori.
3	nama	VARCHAR	Nama asli dokumen yang diunggah beserta ekstensinya.
4	ukuran	BIGINT	Kapasitas ukuran berkas dalam satuan byte.
5	id_manifest	VARCHAR	Kunci referensi (<i>foreign indicator</i>) yang menghubungkan entitas berkas di

No	Nama Kolom	Tipe Data	Keterangan
			API dengan struktur Object Manifest di dalam Vault Core (BadgerDB).
6	dibuat_pada	TIMESTAMP	Waktu pengunggahan berkas berhasil diselesaikan.
7	dihapus_pada	TIMESTAMP	Penanda <i>soft delete</i> untuk menyembunyikan berkas dari dasbor pengguna tanpa menghapus referensi manifes fisik di Vault Core.

b. Basis Data Key-Value

Sistem mengimplementasikan basis data non-relasional berjenis *key-value* menggunakan BadgerDB di dalam lingkungan *Vault Core*. Penulis merancang skema *key-value* untuk mengelola metadata fisik berkas tanpa bergantung pada struktur tabel relasional. Basis data *key-value* mengelompokkan manajemen metadata ke dalam tiga struktur data utama secara terpisah.

1) Struktur Data Object Manifest

Struktur data pertama adalah *Object Manifest* yang bertugas mengelola metadata utama dokumen. Sistem menggunakan format kunci spesifik berupa manifest:<file_hash> untuk memetakan urutan potongan data. Atribut JSON menyimpan daftar nilai *hash* BLAKE3 penyusun dokumen, total kapasitas keseluruhan berkas dalam satuan *byte*, serta stempel waktu saat sistem membentuk dan mengunci dokumen manifes.

2) Struktur Data Chunk Reference

Struktur data kedua adalah *Chunk Reference* yang berfungsi mengelola metadata fisik potongan data secara individual. Sistem menerapkan nilai *hash* kriptografi BLAKE3 sebagai kunci pencarian utama, sementara objek JSON menyimpan metrik untuk memfasilitasi algoritma deduplikasi otomatis. Atribut pada objek JSON mencakup ukuran kapasitas potongan data, jumlah dokumen manifes aktif yang merujuk pada potongan data spesifik, serta status penguncian retensi keamanan untuk melacak transisi perlindungan data.

3) Struktur Data Upload Session

Struktur data ketiga adalah *Upload Session* yang berperan memantau status pengunggahan berkas secara langsung. Sistem menjadikan identitas sesi sementara sebagai kunci pencarian untuk menjamin sifat idempoten dan mencegah duplikasi transmisi saat klien mengalami kegagalan jaringan. Objek JSON pada struktur sesi merekam daftar nilai *hash* dari potongan data yang berhasil sistem pindahkan ke area persinggahan, beserta batas waktu kedaluwarsa sebelum sistem mengeksekusi pembersihan otomatis terhadap data sementara.